IMPERIAL

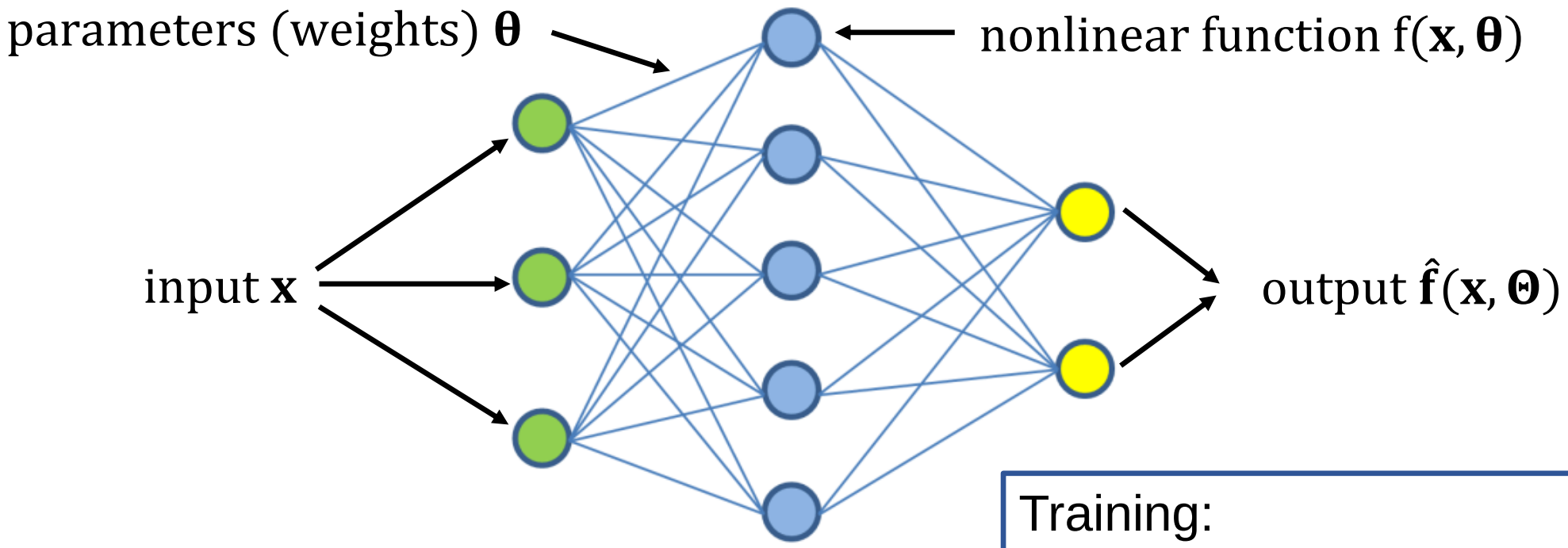
Department of Chemical Engineering

Machine Learning for Chemical Engineering

Lecture 2a – Neural Networks

Recall

- Possibly the most popular nonlinear regression technique.
- NNs are particularly well suited for large datasets in very large dimensional spaces.
- 3-minute explanation:



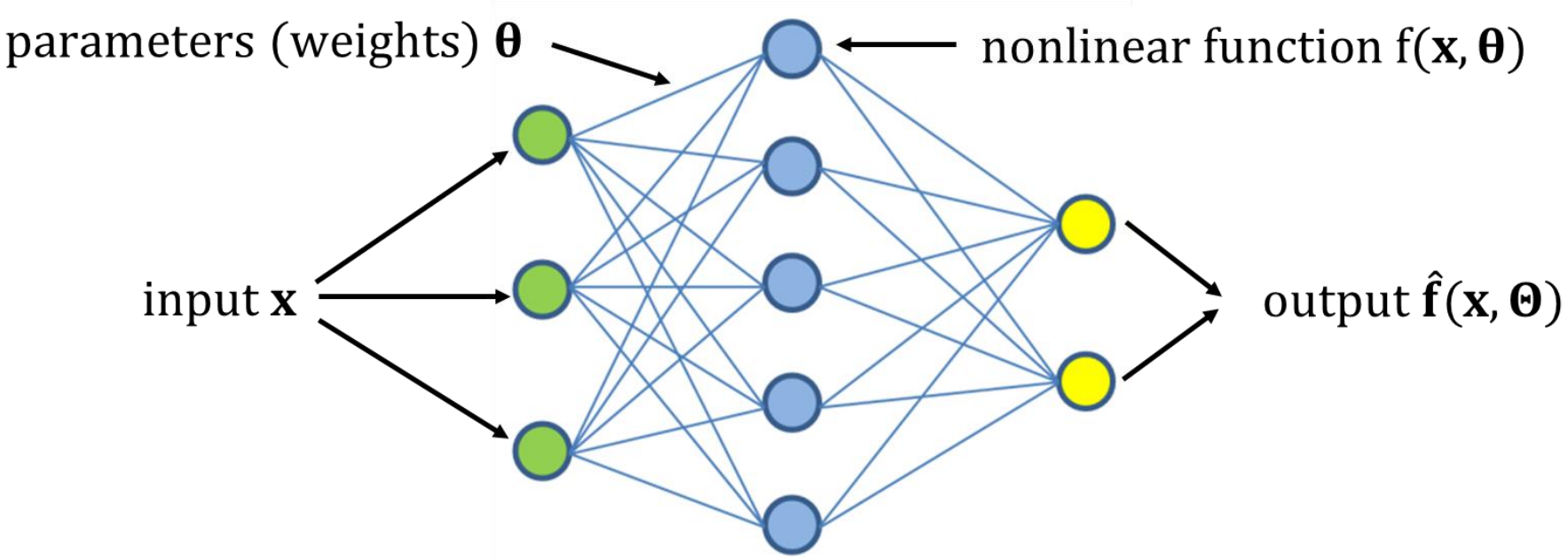
NNs are 'simply' nonlinear functions

Training:

$$\min_{\boldsymbol{\Theta}} \frac{1}{n_d} \sum_{i=1}^{n_d} \left(\hat{f}(\mathbf{x}^{(i)}, \boldsymbol{\Theta}) - y^{(i)} \right)^2$$

Today

- We will look at neural networks.
- Possibly the most popular nonlinear regression technique.
- Particularly well suited for large datasets in very large dimensional spaces.



Data pre-processing (standardization)

Data pre-processing (standardization)

We have an example with 151 points in 2-dimensions $\mathbf{x} \in \mathbb{R}^2$

$$X = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(151)}\right)^T \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ \vdots & \vdots \\ x_1^{(151)} & x_2^{(151)} \end{bmatrix}, \qquad X \in \mathbb{R}^{151\times 2}$$

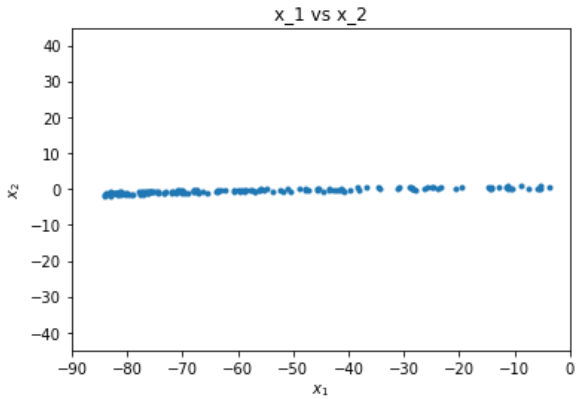
First step for any ML analysis (good practice) is **standardizing** the data:

$$\bar{x}_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{\sigma_j}$$

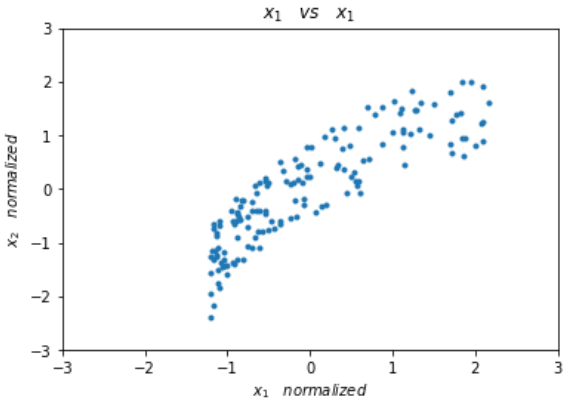
With

$$\mu_j = \frac{1}{151}\sum_{i=1}^{151} x_j^{(i)}, \qquad \sigma_j = \sqrt{\frac{\sum_{i=1}^{151}\left(x_j^{(i)} - \mu_j\right)^2}{150}}$$

Without standardization



With standardization



Data pre-processing (standardization)

We have an example with 151 points in 2-dimensions $\mathbf{x} \in \mathbb{R}^2$

$$X = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(151)}\right)^T \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ \vdots & \vdots \\ x_1^{(151)} & x_2^{(151)} \end{bmatrix}, \qquad X \in \mathbb{R}^{151\times 2}$$

First step for any ML analysis (good practice) is **standardizing** the data:

$$\bar{x}_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{\sigma_j}$$

With

$$\mu_j = \frac{1}{151}\sum_{i=1}^{151} x_j^{(i)}, \qquad \sigma_j = \sqrt{\frac{\sum_{i=1}^{151}\left(x_j^{(i)} - \mu_j\right)^2}{150}}$$

$$\bar{X} = \begin{bmatrix} \left(\bar{\mathbf{x}}^{(1)}\right)^T \\ \vdots \\ \left(\bar{\mathbf{x}}^{(151)}\right)^T \end{bmatrix} = \begin{bmatrix} \bar{x}_1^{(1)} & \bar{x}_2^{(1)} \\ \vdots & \vdots \\ \bar{x}_1^{(151)} & \bar{x}_2^{(151)} \end{bmatrix}$$

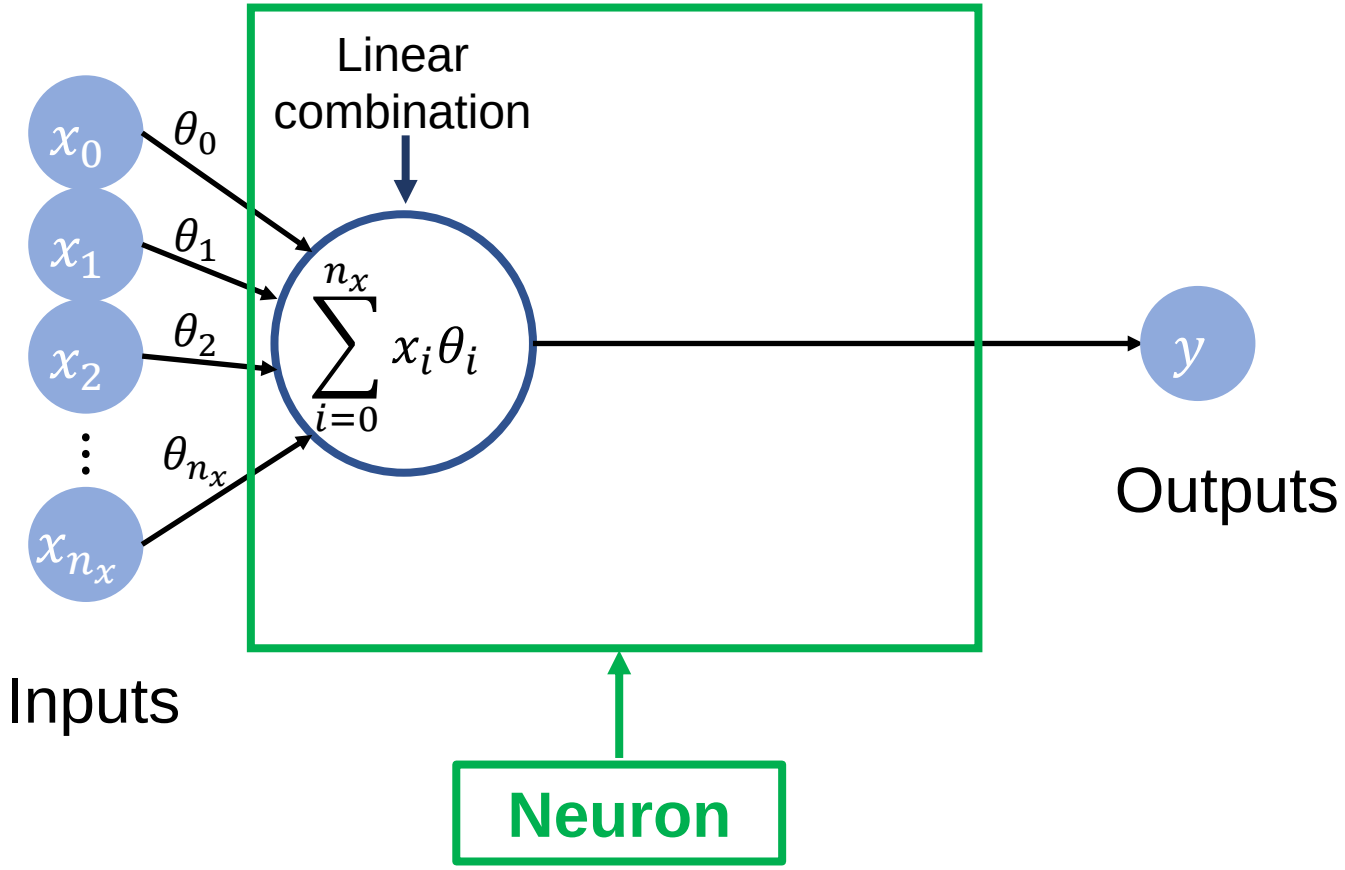
We assume all our data is standardized from now on $\mathbf{x} \to \bar{\mathbf{x}}$

Linear regression as a neuron

Supervised Learning – Linear regression

- Visualizing **linear regression as a neural network**

$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$

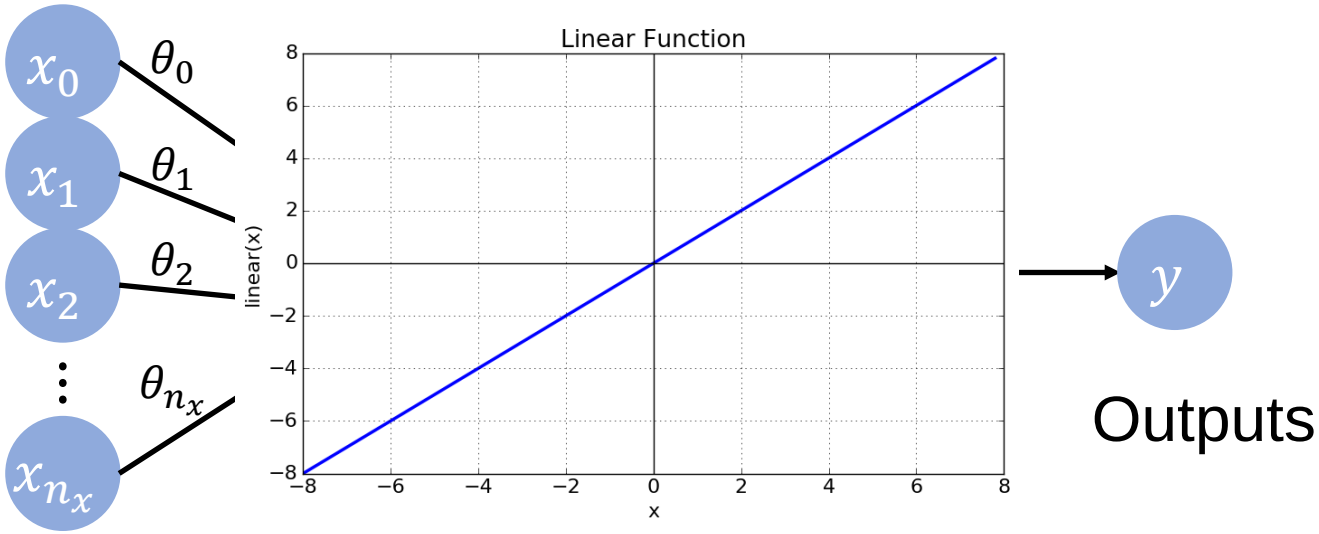


$$y = \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{n_x} x_{n_x} + \theta_0$$

Supervised Learning – Linear regression

- Visualizing **linear regression as a neural network**

$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$



Inputs

Outputs

Neuron

$$y = \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{n_x} x_{n_x} + \theta_0$$

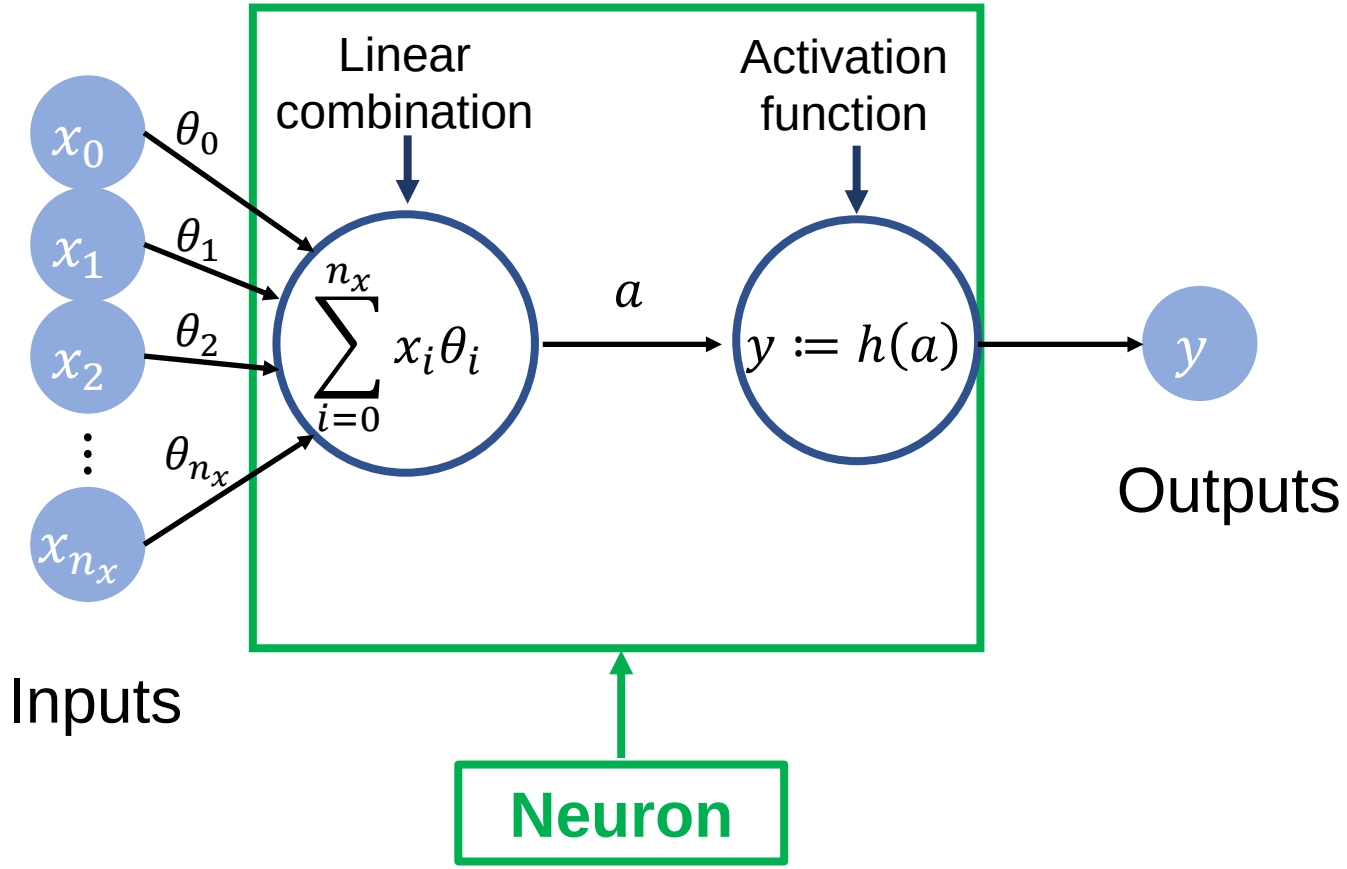
Supervised Learning – The ANN unit: Perceptron

- From linear regression to a neural network
- Using **sigmoid** as activation function

$$h(a) \coloneqq \text{sigmoid}(a) = \frac{1}{1+\exp(-a)}$$

$$\boxed{\frac{1}{1+\exp(-\sum_{i=0}^{n_x} x_i\theta_i)}}$$

$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$



$$\boxed{y = h(\theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{n_x} x_{n_x} + \theta_0)}$$

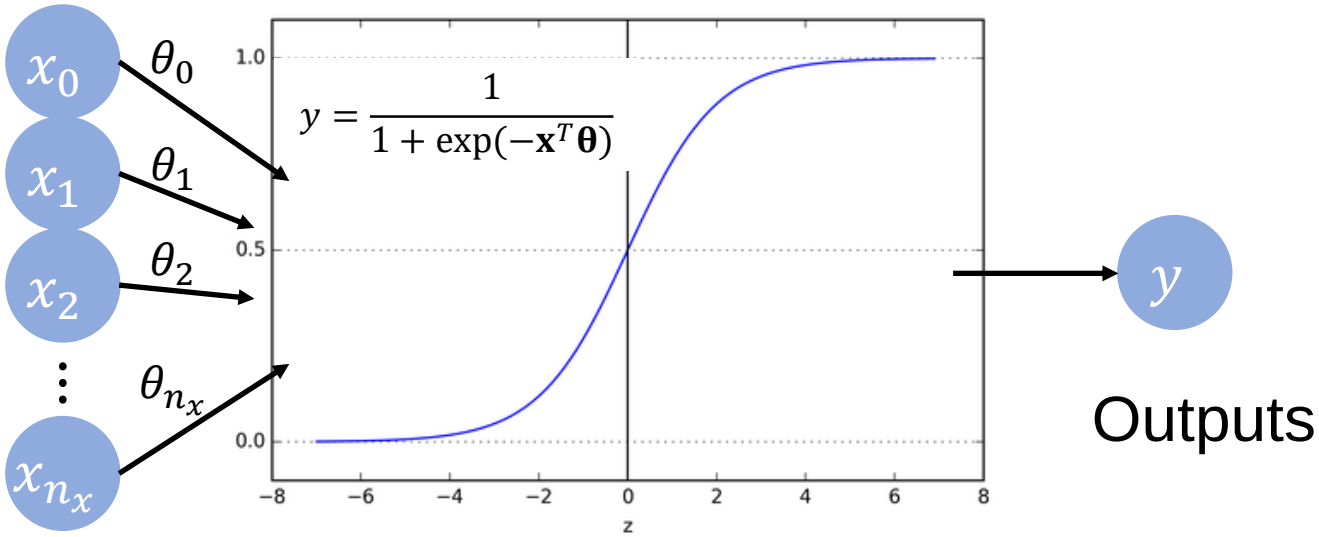
Supervised Learning – The ANN unit: Perceptron

- From linear regression to a neural network
- Using **sigmoid** as activation function
- Mostly used in classification $y \in [0,1]$

$$h(a) \coloneqq \text{sigmoid}(a) = \frac{1}{1+\exp(-a)}$$

$$\boxed{\frac{1}{1+\exp(-\sum_{i=0}^{n_x} x_i\theta_i)}}$$

$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$



Outputs

Inputs

Neuron

$$\boxed{y = h(\theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{n_x} x_{n_x} + \theta_0)}$$

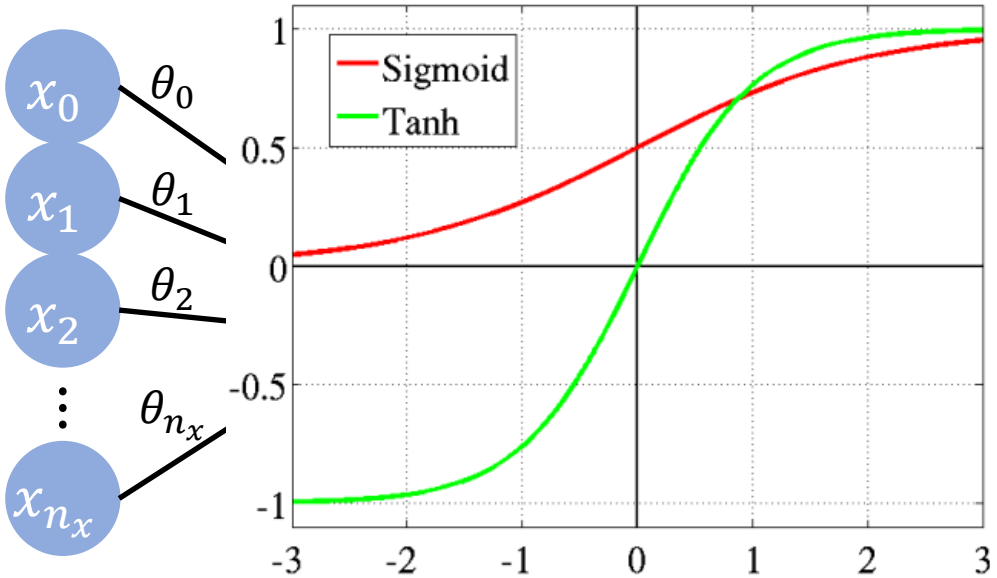
Supervised Learning – The ANN unit: Perceptron

- From linear regression to a neural network
- Using **tanh** as activation function
- Mostly used in regression $y \in [-1,1]$

$$h(a) := \tanh(a) = \frac{2}{1+\exp(-2a)} - 1$$

$$h(\mathbf{x}^T\boldsymbol{\theta}) := \tanh(\mathbf{x}^T\boldsymbol{\theta}) = \frac{2}{1+\exp(-2\mathbf{x}^T\boldsymbol{\theta})} - 1$$

$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$



y

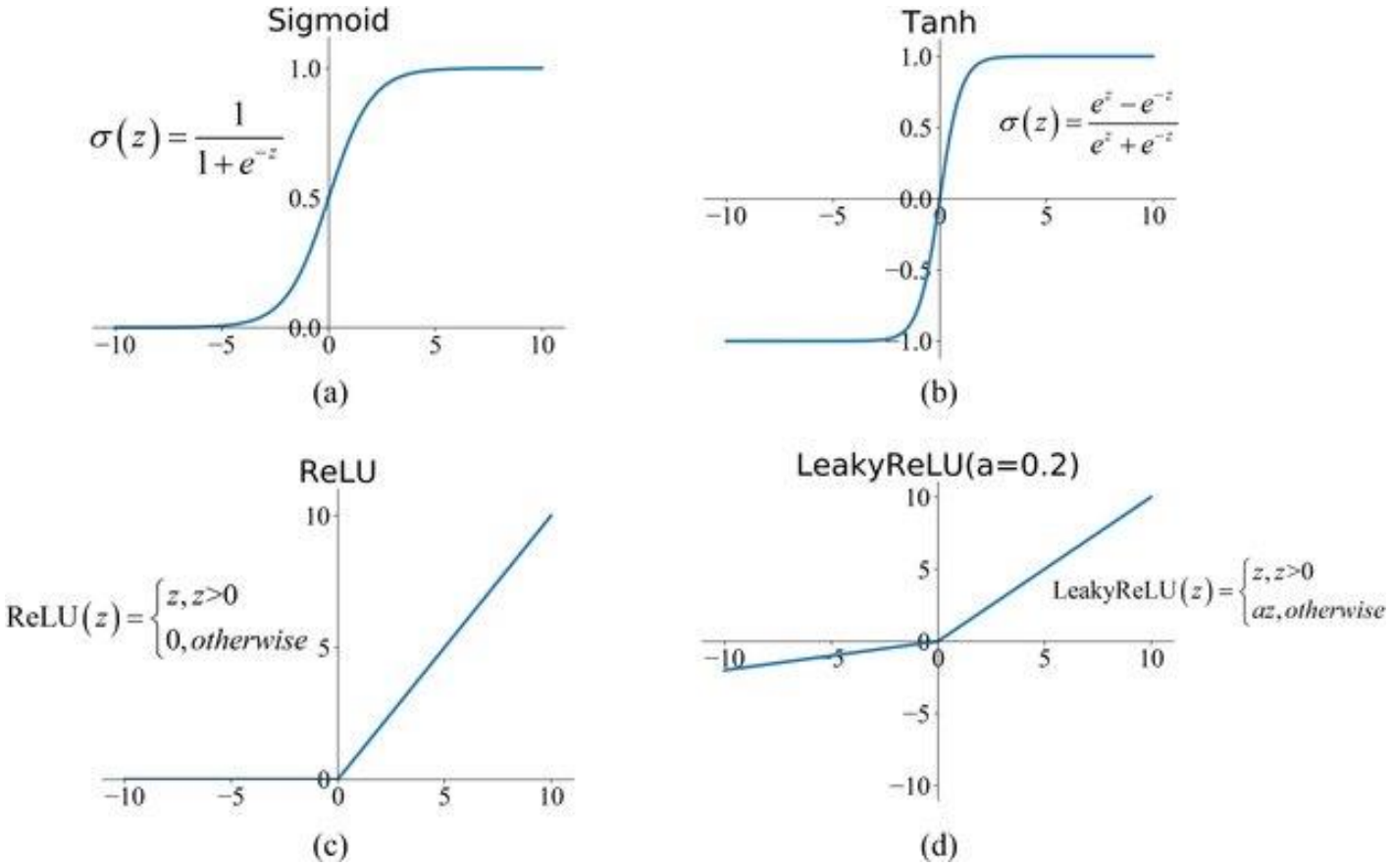
Outputs

Inputs

Neuron

$$y = h(\theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{n_x} x_{n_x} + \theta_0)$$

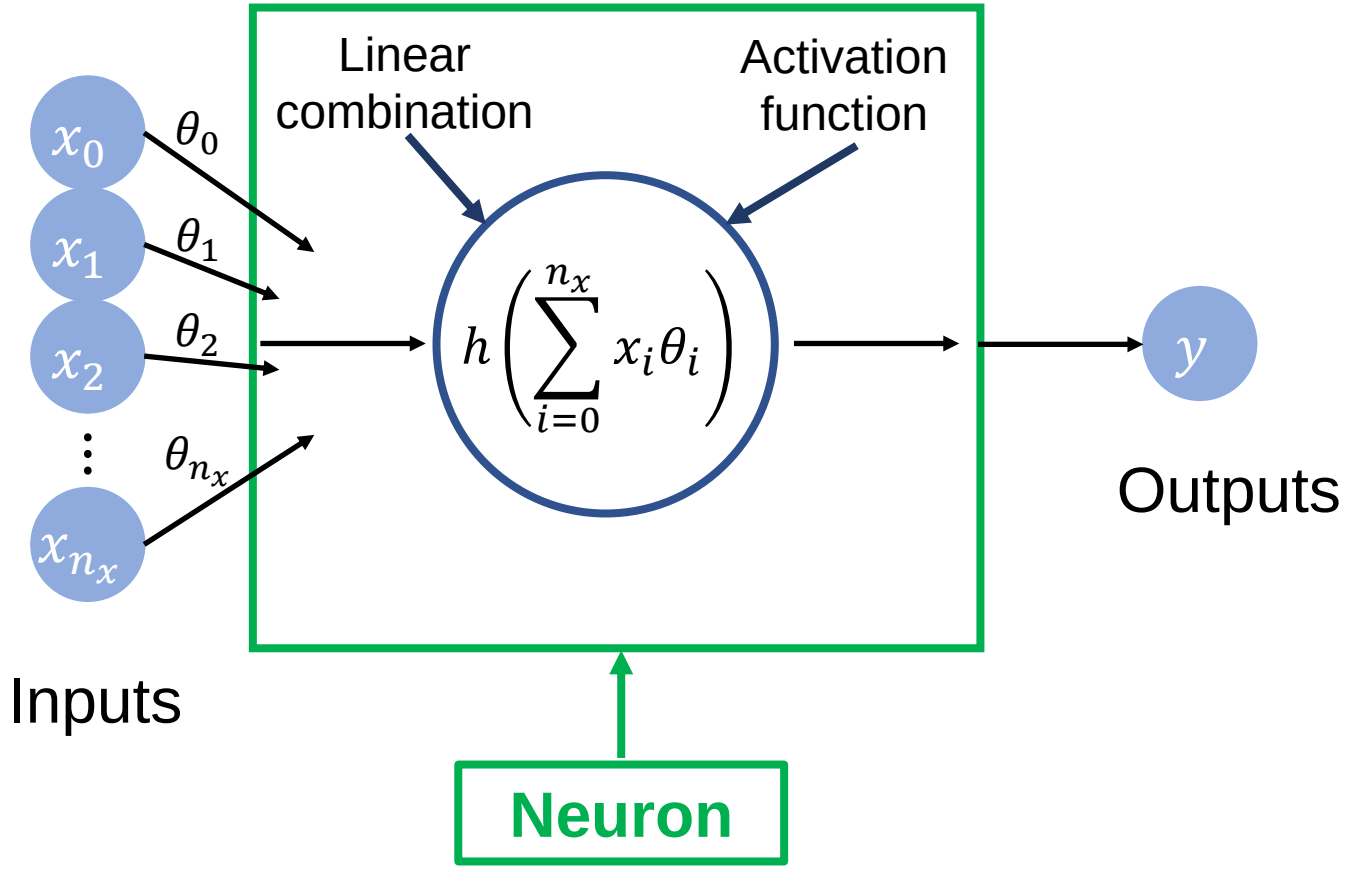
Supervised Learning – Linear regression



Supervised Learning – The ANN unit: Perceptron

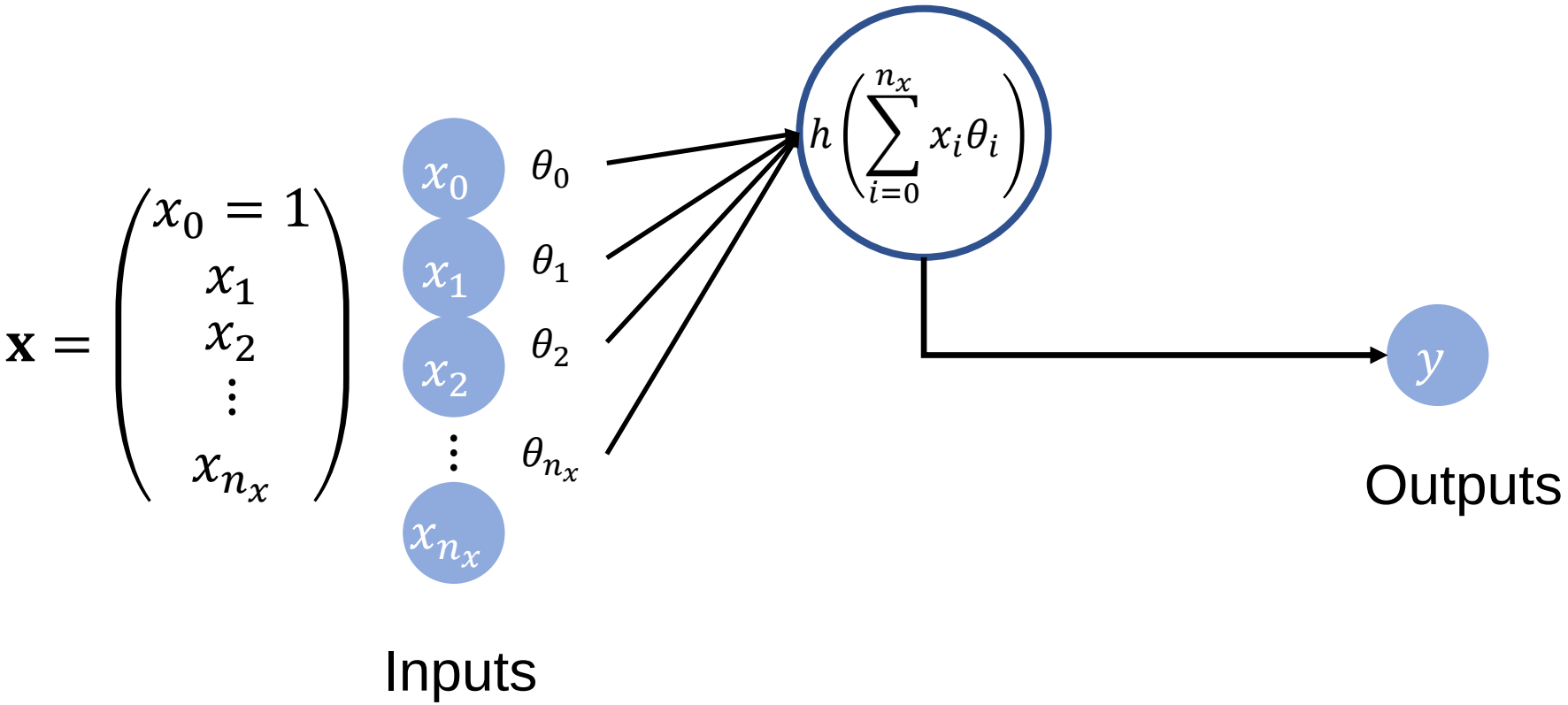
- From linear regression to a neural network

$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$



Supervised Learning – Feedforward Neural Network

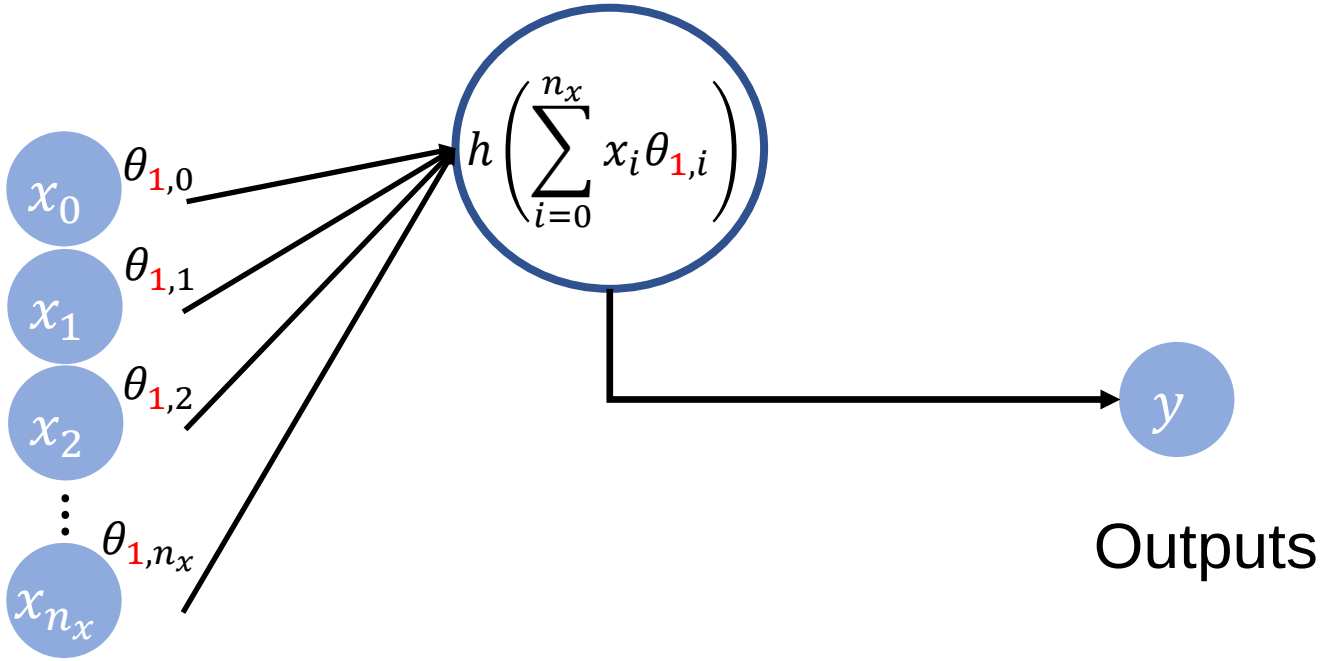
- From linear regression to a neural network



Supervised Learning – Feedforward Neural Network

- From linear regression to a neural network

$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$



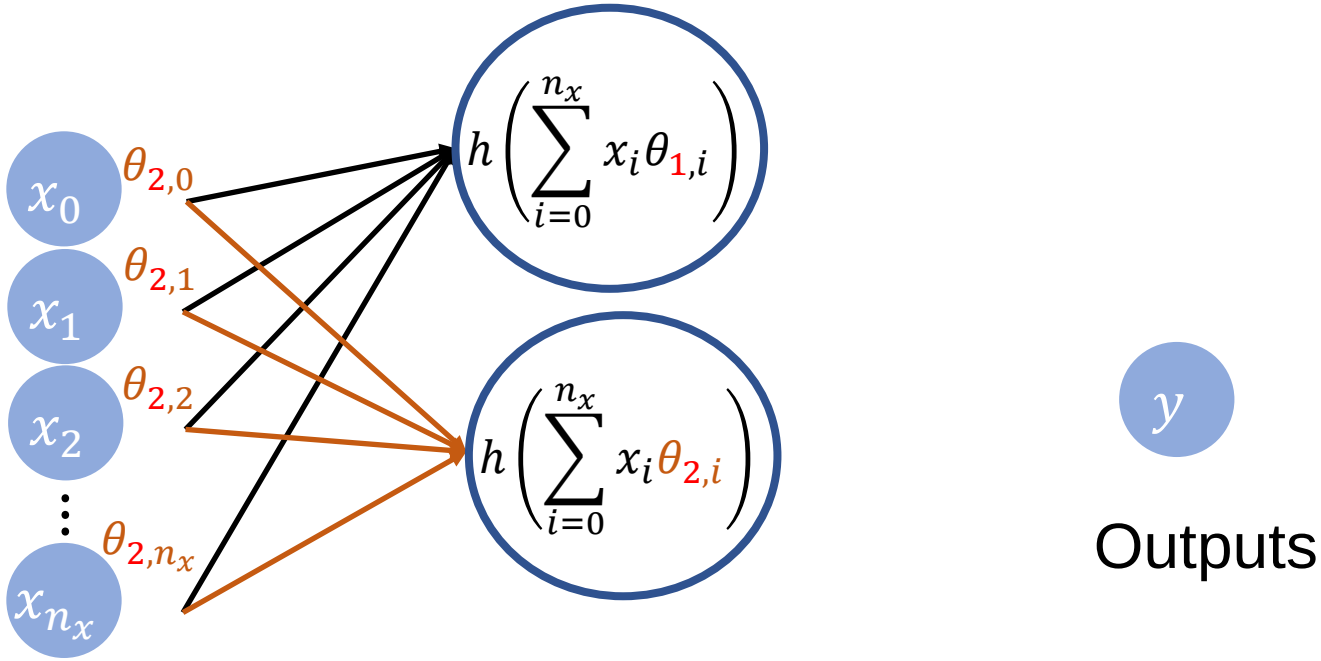
Inputs

Outputs

Supervised Learning – Feedforward Neural Network

- From linear regression to a neural network

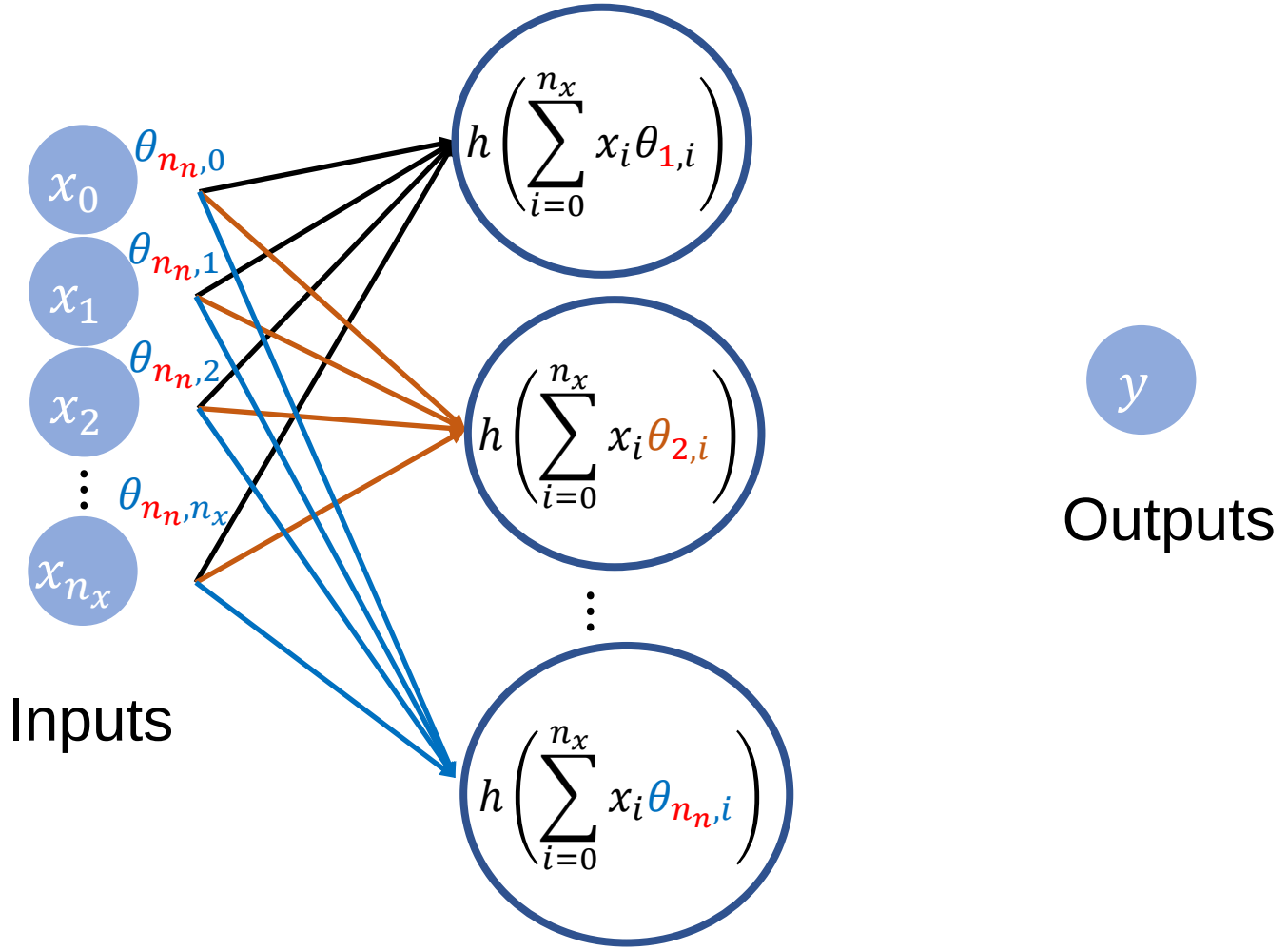
$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$



Supervised Learning – Feedforward Neural Network

- From linear regression to a neural network

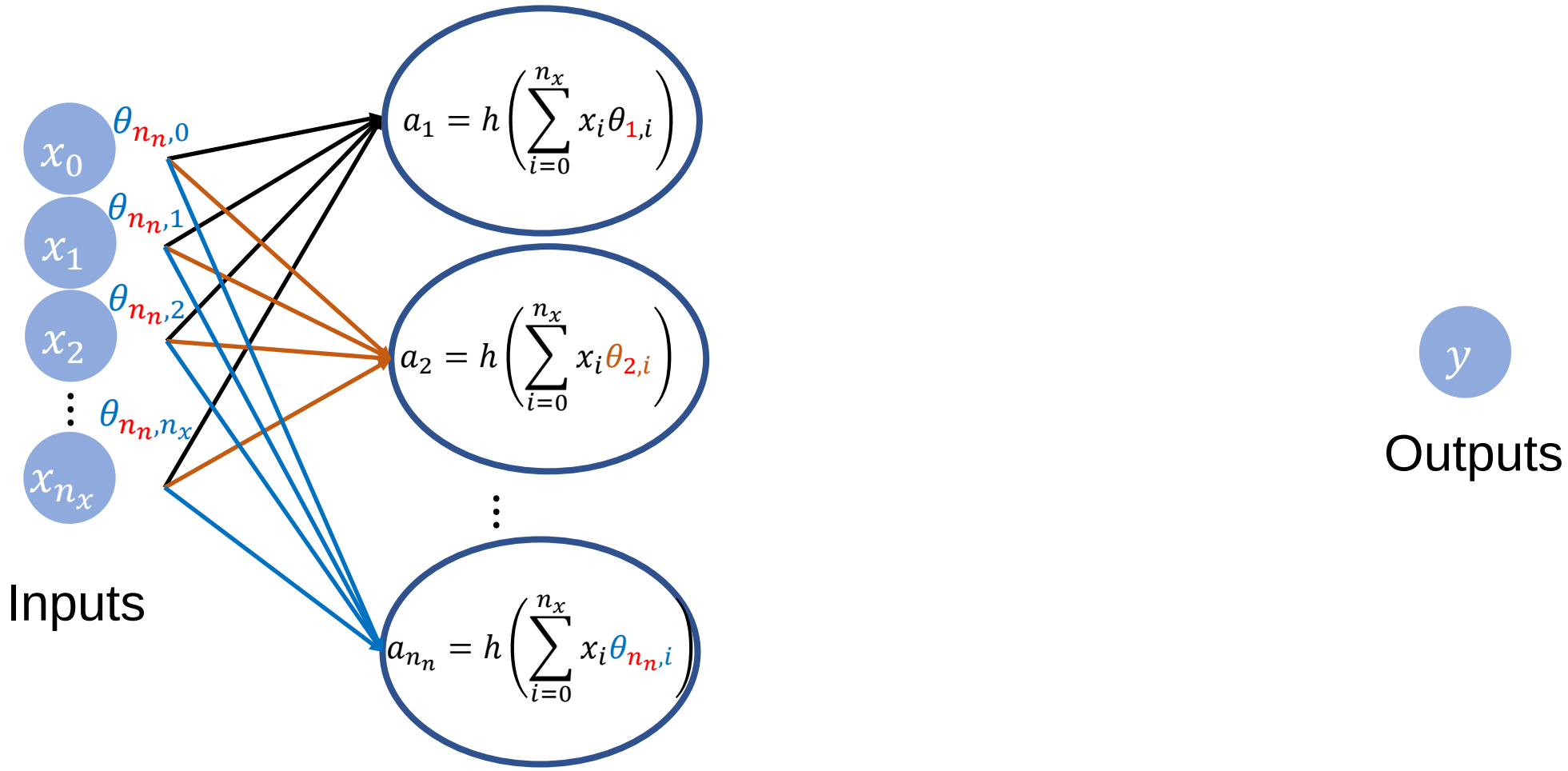
$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$



Supervised Learning – Feedforward Neural Network

- From linear regression to a neural network

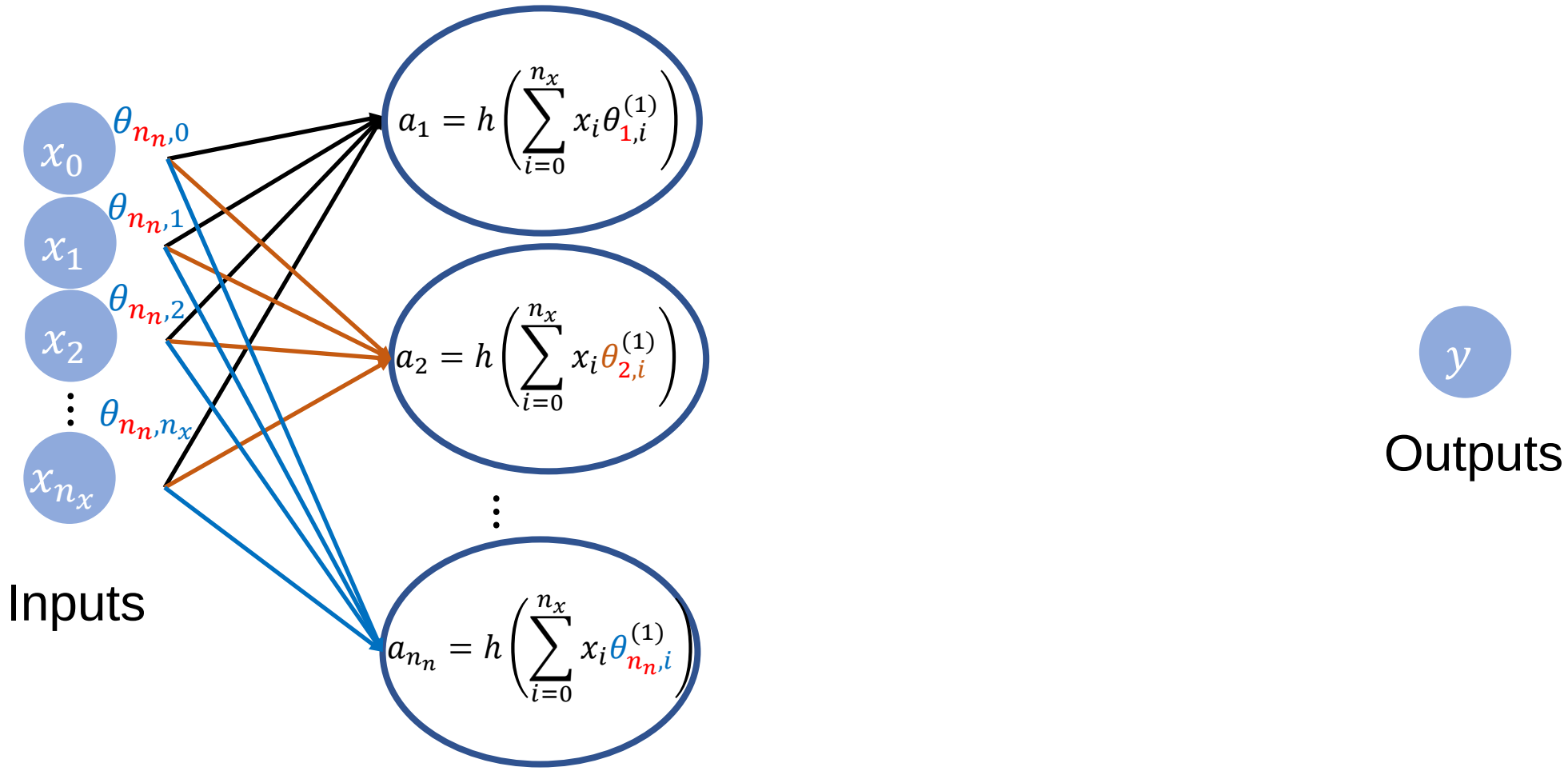
$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$



Supervised Learning – Feedforward Neural Network

- From linear regression to a neural network

$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$

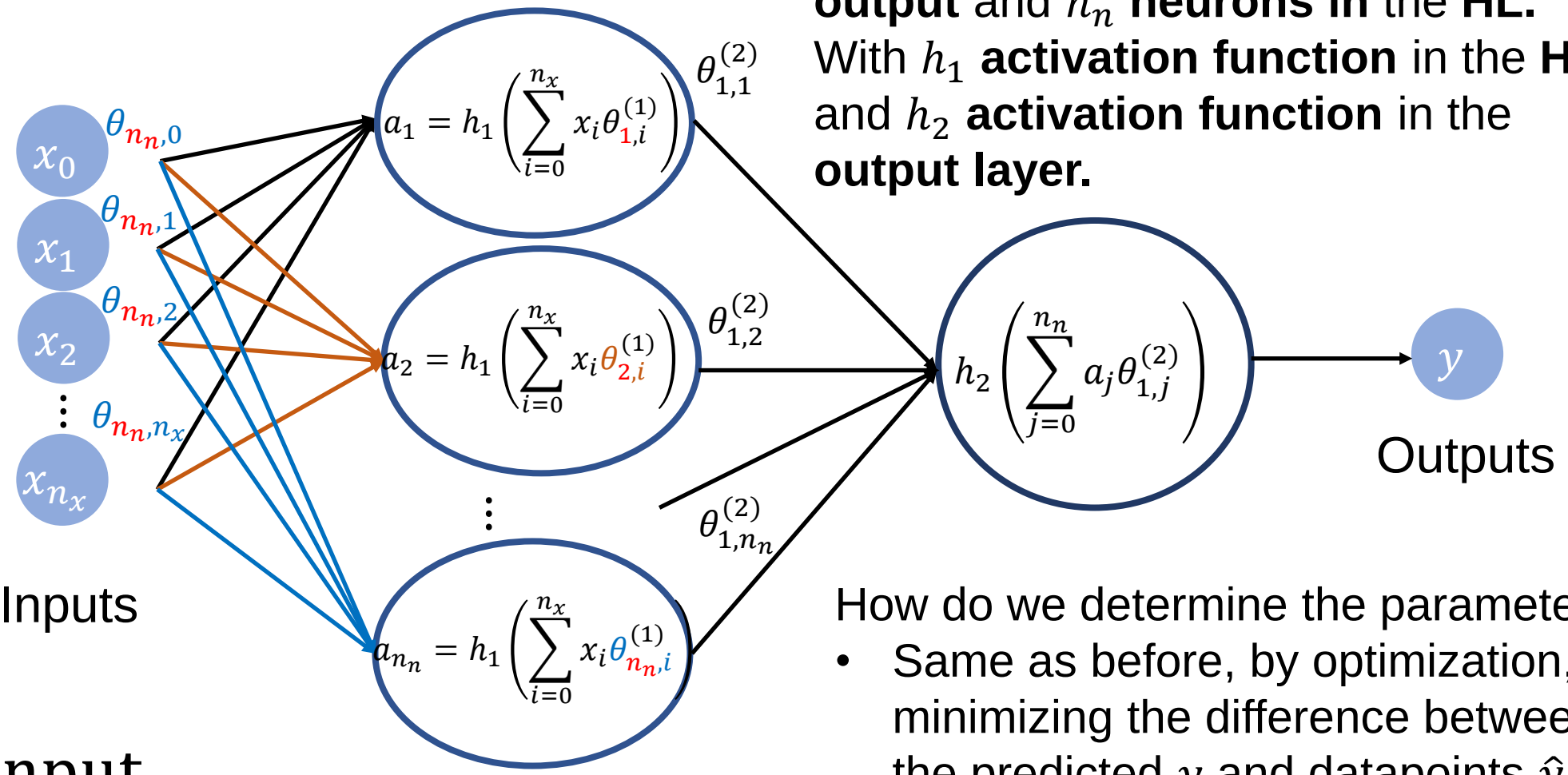


Supervised Learning – Feedforward Neural Network

- From linear regression to a neural network

This is a feedforward NN with **one hidden layer (HL),** with n_x **inputs, 1 output** and n_n **neurons in** the **HL.** With h_1 **activation function** in the **HL** and h_2 **activation function** in the **output layer.**

$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$



$$\theta^{(\text{layer})}_{\text{neuron, input}}$$

How do we determine the parameters?

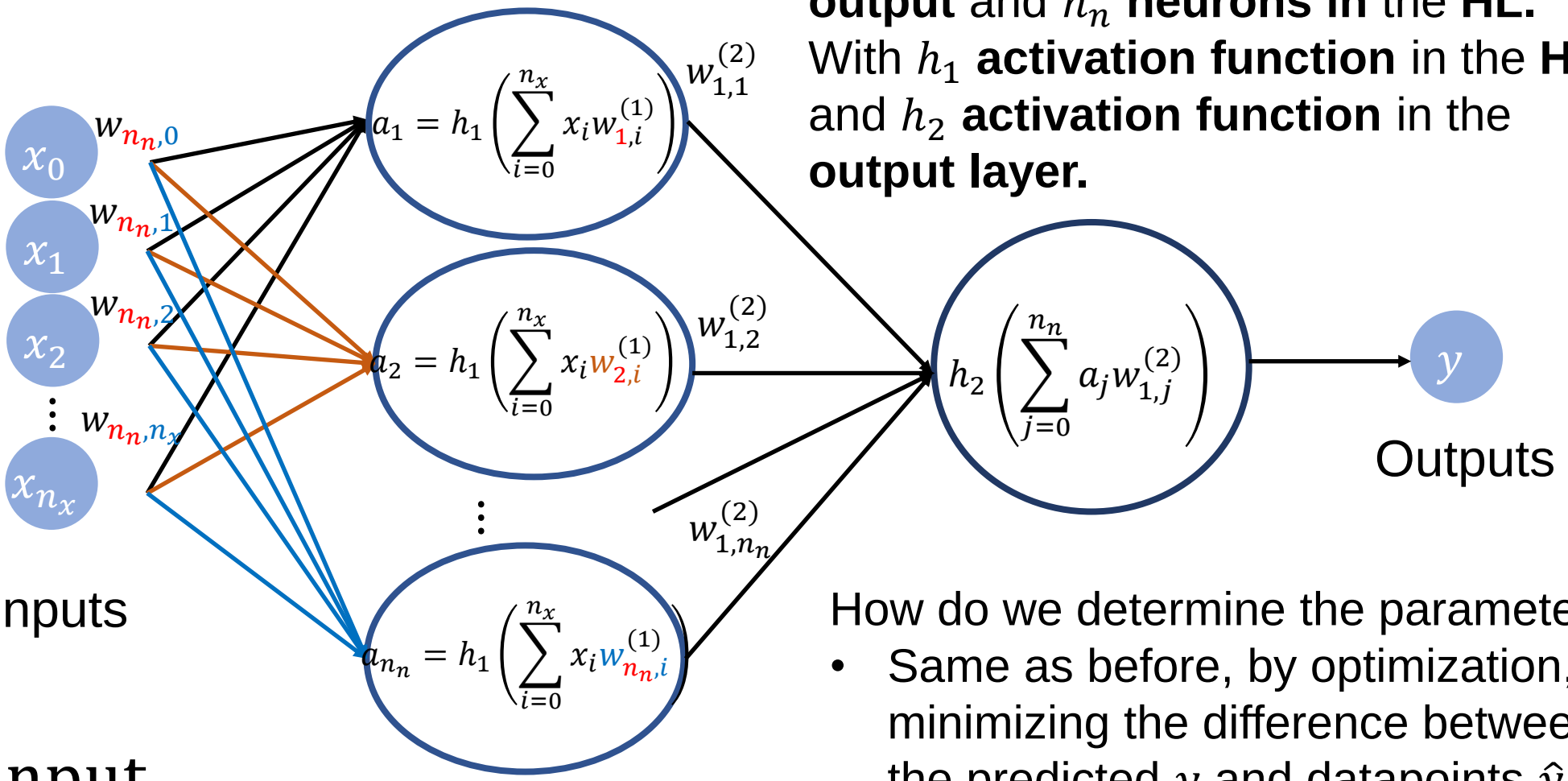
- Same as before, by optimization, by minimizing the difference between the predicted y and datapoints $\hat{y}$

Supervised Learning – Feedforward Neural Network

- From linear regression to a neural network

This is a feedforward NN with **one hidden layer (HL),** with n_x **inputs, 1 output** and n_n **neurons in** the **HL.** With h_1 **activation function** in the **HL** and h_2 **activation function** in the **output layer.**

$$\mathbf{x} = \begin{pmatrix} x_0 = 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{n_x} \end{pmatrix}$$



$$w_{\text{neuron, input}}^{(\text{layer})}$$

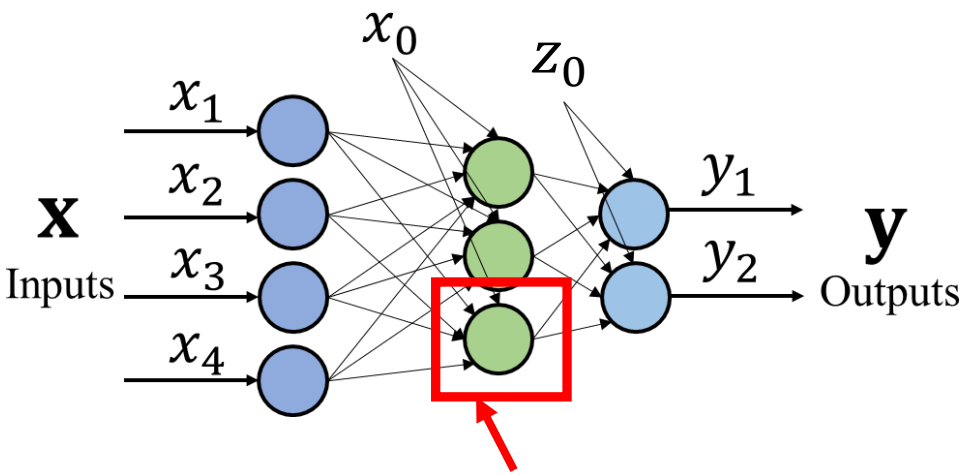
How do we determine the parameters?

- Same as before, by optimization, by minimizing the difference between the predicted y and datapoints $\hat{y}$

‘Learning’
(Parameter estimation)

Learning (Parameter estimation)

Given labelled 'training' data $\mathcal{D} = \left\{\left(\mathbf{y}^{(1)}, \mathbf{x}^{(1)}\right), \left(\mathbf{y}^{(2)}, \mathbf{x}^{(2)}\right), \ldots, \left(\mathbf{y}^{(n_d)}, \mathbf{x}^{(n_d)}\right)\right\} = \{(\mathbf{X}, \mathbf{Y})\}$



$$\mathbf{w}_3^{(1)} = \left\{w_{3,0}^{(1)}, \ldots, w_{3,4}^{(1)}\right\}$$

$$z_3^{(1)} = h^{(1)}\left(a_3^{(1)}\right) = h^{(1)}\left(\mathbf{x}^T \mathbf{w}_3^{(1)}\right)$$

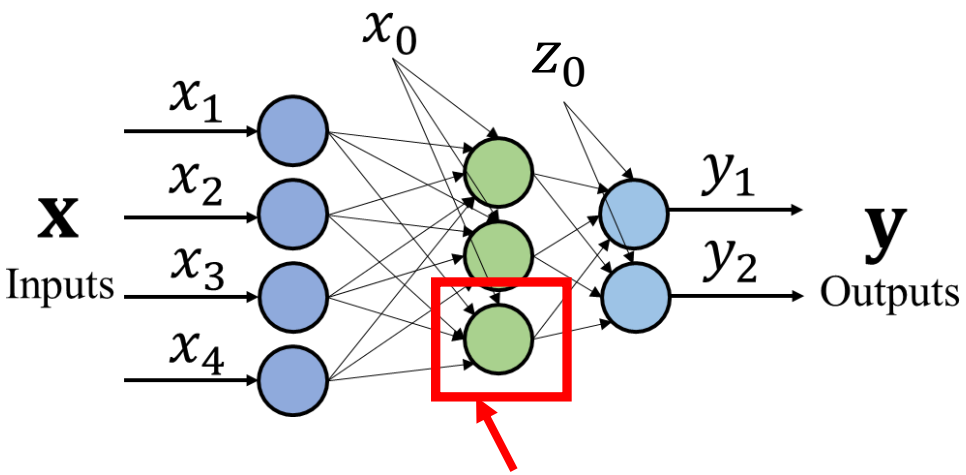
* slight change of notation from previous slides with z being the output of each neuron and $a = \mathbf{x}^T\mathbf{w}$

* x_0 and z_0 are ones: the bias terms.

Learning (Parameter estimation)

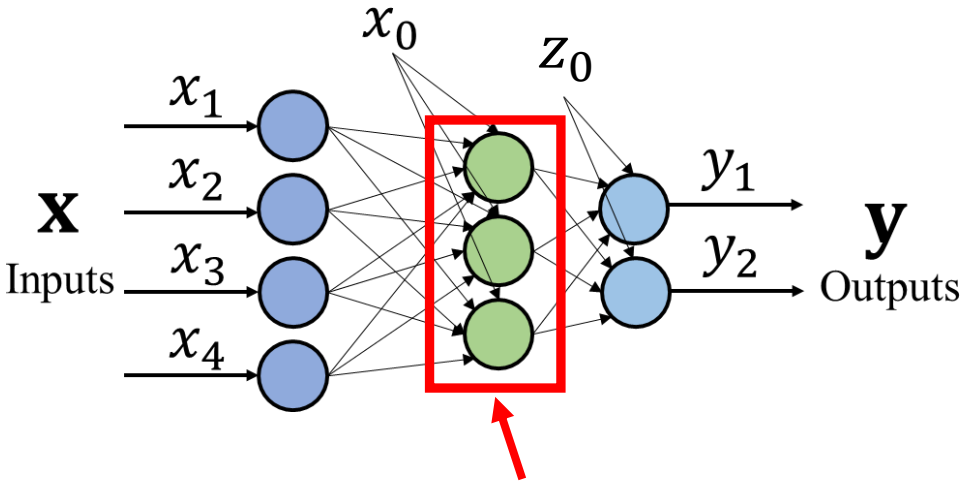
Given labelled 'training' data $\mathcal{D} = \left\{\left(\mathbf{y}^{(1)}, \mathbf{x}^{(1)}\right), \left(\mathbf{y}^{(2)}, \mathbf{x}^{(2)}\right), \dots, \left(\mathbf{y}^{(n_d)}, \mathbf{x}^{(n_d)}\right)\right\} = \{(\mathbf{X}, \mathbf{Y})\}$

Notation: We can formulate an optimisation problem to estimate the parameters (weights) $W^{(1)}, \dots, W^{(n_{hl})}$, n_{hl} is number of hidden layers.



$$\mathbf{w}_3^{(1)} = \left\{w_{3,0}^{(1)}, \dots, w_{3,4}^{(1)}\right\}$$

$$z_3^{(1)} = h^{(1)}\left(a_3^{(1)}\right) = h^{(1)}\left(\mathbf{x}^T \mathbf{w}_3^{(1)}\right)$$



$$W^{(1)} = \left\{w_{1,0}^{(1)}, w_{1,1}^{(1)}, \dots w_{2,1}^{(1)}, \dots, w_{3,1}^{(1)}, \dots, w_{3,4}^{(1)}\right\}$$

$$\mathbf{z}^{(1)} = \left\{1, z_1^{(1)}, z_2^{(1)}, z_3^{(1)}\right\}$$

z_0

Learning (Parameter estimation)

Given labelled 'training' data $\mathcal{D} = \{(\mathbf{y}^{(1)}, \mathbf{x}^{(1)}), (\mathbf{y}^{(2)}, \mathbf{x}^{(2)}), \dots, (\mathbf{y}^{(n_d)}, \mathbf{x}^{(n_d)})\} = \{(\mathbf{X}, \mathbf{Y})\}$

Notation: We can formulate an optimisation problem to estimate the parameters (weights) $W^{(1)}, \dots, W^{(n_{hl})}$, n_{hl} is number of hidden layers.

$\mathbf{X} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_d)}], \mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$ and $\mathbf{Y} = [\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(n_d)}], \mathbf{y}^{(i)} \in \mathbb{R}^{n_y}$. The optimization is as follows:

$$\min_{W^{(1)},\dots,W^{(n_{hl})}} \boxed{\mathcal{L}(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y})}$$

A likelihood function measures how close model predictions and data are.

Where $\mathcal{L}(\cdot)$ is some likelihood function.

Learning (Parameter estimation)

Given labelled 'training' data $\mathcal{D} = \{(\mathbf{y}^{(1)}, \mathbf{x}^{(1)}), (\mathbf{y}^{(2)}, \mathbf{x}^{(2)}), \dots, (\mathbf{y}^{(n_d)}, \mathbf{x}^{(n_d)})\} = \{(\mathbf{X}, \mathbf{Y})\}$

Notation: We can formulate an optimisation problem to estimate the parameters (weights) $W^{(1)}, \dots, W^{(n_{hl})}$, n_{hl} is number of hidden layers.

$\mathbf{X} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_d)}], \mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$ and $\mathbf{Y} = [\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(n_d)}], \mathbf{y}^{(i)} \in \mathbb{R}^{n_y}$. The optimization is as follows:

$$\min_{W^{(1)},\dots,W^{(n_{hl})}} \quad \mathcal{L}(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y})$$

Where $\mathcal{L}(\cdot)$ is some likelihood function.

$$\text{regression} \quad \frac{1}{n_y}\frac{1}{n_d}\sum_{i=1}^{n_y}\sum_{j=1}^{n_d}\Big(\underbrace{y_i^{(j)}}_{\text{data}} - \underbrace{\mathcal{N}_{\mathcal{N}}(W^{(1:n_{hl})}; \mathbf{X}, \mathbf{Y})_i^{(j)}}_{\text{Model prediction}}\Big)^2$$

Learning (Parameter estimation)

Given labelled 'training' data $\mathcal{D} = \left\{\left(\mathbf{y}^{(1)}, \mathbf{x}^{(1)}\right), \left(\mathbf{y}^{(2)}, \mathbf{x}^{(2)}\right), \dots, \left(\mathbf{y}^{(n_d)}, \mathbf{x}^{(n_d)}\right)\right\} = \{(\mathbf{X}, \mathbf{Y})\}$

Notation: We can formulate an optimisation problem to estimate the parameters (weights) $W^{(1)}, \dots, W^{(n_{hl})}$, n_{hl} is number of hidden layers.

$\mathbf{X} = \left[\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_d)}\right], \mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$ and $\mathbf{Y} = \left[\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(n_d)}\right], \mathbf{y}^{(i)} \in \mathbb{R}^{n_y}$. The optimization is as follows:

$$\min_{W^{(1)}, \dots, W^{(n_{hl})}} \quad \mathcal{L}\left(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y}\right)$$

Where $\mathcal{L}(\cdot)$ is some likelihood function.

$$\text{regression} \quad \frac{1}{n_y}\frac{1}{n_d}\sum_{i=1}^{n_y}\sum_{j=1}^{n_d}\left(y_i^{(j)} - \mathcal{N}_{\mathcal{N}}\left(W^{(1:n_{hl})}; \mathbf{X}, \mathbf{Y}\right)_i^{(j)}\right)^2$$

$\mathcal{N}_{\mathcal{N}}\left(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y}\right)$ is the neural network.

$$\min_{W^{(1)}, \dots, W^{(n_{hl})}} \quad \frac{1}{n_y}\frac{1}{n_d}\sum_{i=1}^{n_y}\sum_{j=1}^{n_d}\left(y_i^{(j)} - \mathcal{N}_{\mathcal{N}}\left(W^{(1:n_{hl})}; \mathbf{X}, \mathbf{Y}\right)_i^{(j)}\right)^2$$

Learning (Parameter estimation)

Given labelled 'training' data $\mathcal{D} = \left\{\left(\mathbf{y}^{(1)}, \mathbf{x}^{(1)}\right), \left(\mathbf{y}^{(2)}, \mathbf{x}^{(2)}\right), \dots, \left(\mathbf{y}^{(n_d)}, \mathbf{x}^{(n_d)}\right)\right\} = \{(\mathbf{X}, \mathbf{Y})\}$

Notation: We can formulate an optimisation problem to estimate the parameters (weights) $W^{(1)}, \dots, W^{(n_{hl})}$, n_{hl} is number of hidden layers.

$\mathbf{X} = \left[\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_d)}\right], \mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$ and $\mathbf{Y} = \left[\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(n_d)}\right], \mathbf{y}^{(i)} \in \mathbb{R}^{n_y}$. The optimization is as follows:

$$\min_{W^{(1)}, \dots, W^{(n_{hl})}} \quad \mathcal{L}\left(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y}\right)$$

$\mathcal{N}_{\mathcal{N}}\left(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y}\right)$ is the neural network.

$$\min_{W^{(1)}, \dots, W^{(n_{hl})}} \quad \frac{1}{n_y} \frac{1}{n_d} \sum_{i=1}^{n_y} \sum_{j=1}^{n_d} \left(y_i^{(j)} - \mathcal{N}_{\mathcal{N}}\left(W^{(1:n_{hl})}; \mathbf{X}, \mathbf{Y}\right)_i^{(j)}\right)^2$$

This is a **nonconvex**, and **multimodal** optimisation problem. Generally, first-order gradient-based algorithms are used. Derivatives are calculated through backpropagation which is a special kind of algorithmic differentiation, and the use of first-order methods is due to the dimensionality of the data and the parameters. Popular methods include stochastic gradient descent, Adam, Adagrad, and (L)BFGS

Classification

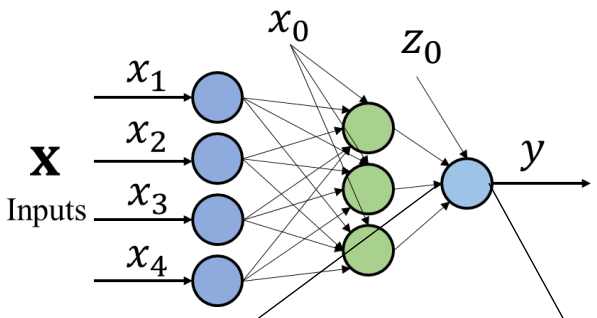
Classification

Given labelled 'training' data $\mathcal{D} = \{(y^{(1)}, \mathbf{x}^{(1)}), (y^{(2)}, \mathbf{x}^{(2)}), \dots, (y^{(n_d)}, \mathbf{x}^{(n_d)})\} = \{(\mathbf{X}, \mathbf{Y})\}$

Weights $W^{(1)}, \dots, W^{(n_{hl})}$, with $\mathbf{X} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_d)}], \mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$ and $\mathbf{Y} = [y^{(1)}, \dots, y^{(n_d)}], y^{(i)} \in \{0,1\}$.

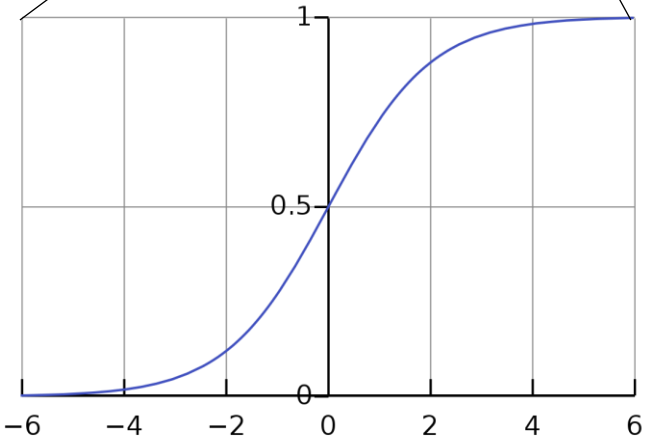
The optimization is as follows:

$$\min_{W^{(1)},\dots,W^{(n_{hl})}} \quad \mathcal{L}(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y})$$



$\mathcal{N}_{\mathcal{N}}(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y})$ is the neural network.

$$\text{classification} \quad -\frac{1}{n_d}\sum_{j=1}^{n_d} y^{(j)}\log\left(\mathcal{N}_{\mathcal{N}}(\cdot)^{(j)}\right) + (1 - y^{(j)})\log\left(1 - \mathcal{N}_{\mathcal{N}}(\cdot)^{(j)}\right)$$



If $y^{(j)} = 1$, then $\mathcal{L} = \log\left(\mathcal{N}_{\mathcal{N}}(\cdot)^{(j)}\right)$, minimum when $\mathcal{N}_{\mathcal{N}}(\cdot)^{(j)} = 1$

If $y^{(j)} = 0$, then $\mathcal{L} = \log\left(1 - \mathcal{N}_{\mathcal{N}}(\cdot)^{(j)}\right)$, minimum when $\mathcal{N}_{\mathcal{N}}(\cdot)^{(j)} = 0$

Here we use a **sigmoid function** as output. Notice $\mathcal{N}_{\mathcal{N}}(\cdot) \in [0,1]$

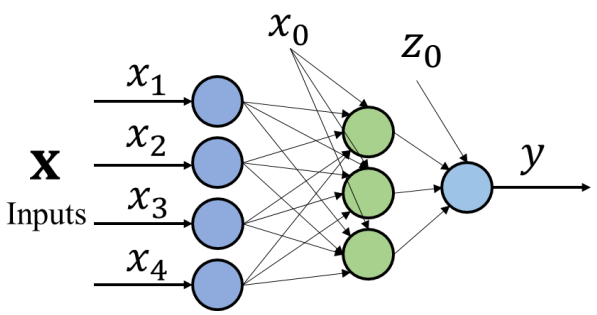
Classification

Given labelled 'training' data $\mathcal{D} = \{(y^{(1)}, \mathbf{x}^{(1)}), (y^{(2)}, \mathbf{x}^{(2)}), \dots, (y^{(n_d)}, \mathbf{x}^{(n_d)})\} = \{(\mathbf{X}, \mathbf{Y})\}$

Weights $W^{(1)}, \dots, W^{(n_{hl})}$, with $\mathbf{X} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_d)}], \mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$ and $\mathbf{Y} = [y^{(1)}, \dots, y^{(n_d)}], y^{(i)} \in \{0,1\}$.

The optimization is as follows:

$$\min_{W^{(1)},\dots,W^{(n_{hl})}} \quad \mathcal{L}(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y})$$



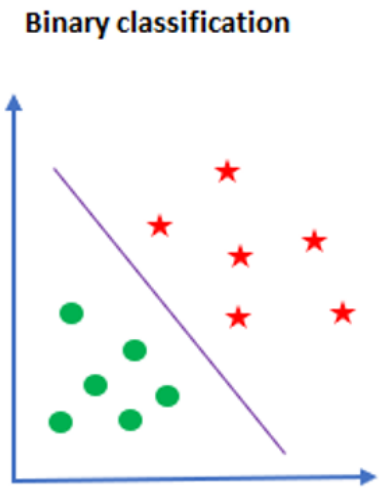
$\mathcal{N}_{\mathcal{N}}(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y})$ is the neural network.

$$\text{classification} \quad -\frac{1}{n_d}\sum_{j=1}^{n_d} y^{(j)}\log\left(\mathcal{N}_{\mathcal{N}}(\cdot)^{(j)}\right) + (1 - y^{(j)})\log\left(1 - \mathcal{N}_{\mathcal{N}}(\cdot)^{(j)}\right)$$

If $y^{(j)} = 1$, then $\mathcal{L} = \log\left(\mathcal{N}_{\mathcal{N}}(\cdot)^{(j)}\right)$, minimum when $\mathcal{N}_{\mathcal{N}}(\cdot)^{(j)} = 1$

If $y^{(j)} = 0$, then $\mathcal{L} = \log\left(1 - \mathcal{N}_{\mathcal{N}}(\cdot)^{(j)}\right)$, minimum when $\mathcal{N}_{\mathcal{N}}(\cdot)^{(j)} = 0$

Here we use a **sigmoid function** as output. Notice $\mathcal{N}_{\mathcal{N}}(\cdot) \in [0,1]$



Multiclass Classification

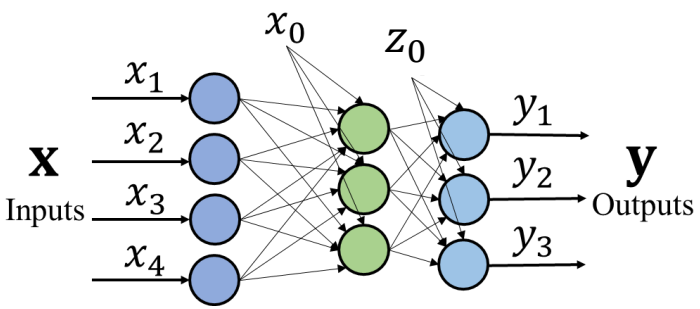
Multiclass Classification

Given labelled 'training' data $\mathcal{D} = \left\{\left(\mathbf{y}^{(1)}, \mathbf{x}^{(1)}\right), \left(\mathbf{y}^{(2)}, \mathbf{x}^{(2)}\right), \dots, \left(\mathbf{y}^{(n_d)}, \mathbf{x}^{(n_d)}\right)\right\} = \{(\mathbf{X}, \mathbf{Y})\}$

Weights $W^{(1)}, \dots, W^{(n_{hl})}$, with $\mathbf{X} = \left[\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_d)}\right], \mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$ and $\mathbf{Y} = \left[\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(n_d)}\right], \mathbf{y}^{(i)} \in \{0,1\}^{n_y}$.

The optimization is as follows:

$$\min_{W^{(1)},\dots,W^{(n_{hl})}} \quad \mathcal{L}\left(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y}\right)$$

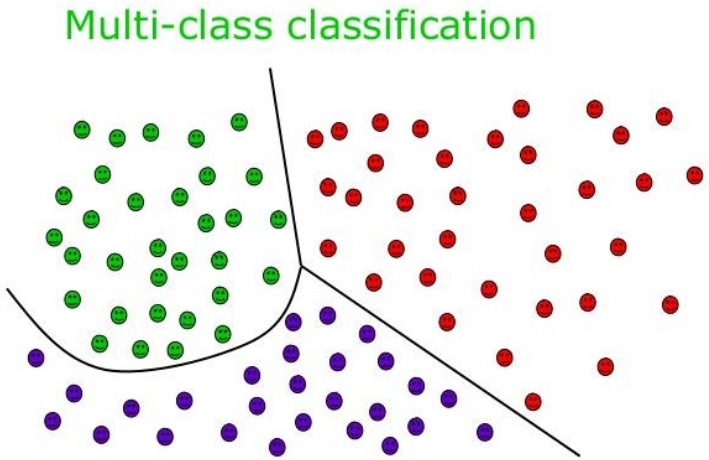


$\mathcal{N}_{\mathcal{N}}\left(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y}\right)$ is the neural network.

$$\text{classification} \quad -\frac{1}{n_y}\frac{1}{n_d}\sum_{i=1}^{n_y}\sum_{j=1}^{n_d} y_i^{(j)} \log\left(\mathcal{N}_{\mathcal{N}}(\cdot)_i^{(j)}\right)$$

Neurons in the last layer are a Softmax

$$\hat{y}_i = \frac{e^{y_i}}{\sum_{j=1}^{n_y} e^{y_j}} \quad \text{or} \quad \hat{y}_i = \frac{e^{\beta y_i}}{\sum_{j=1}^{n_y} e^{\beta y_j}}$$



This is also a very hard problem to solve, just as in the case for regressior

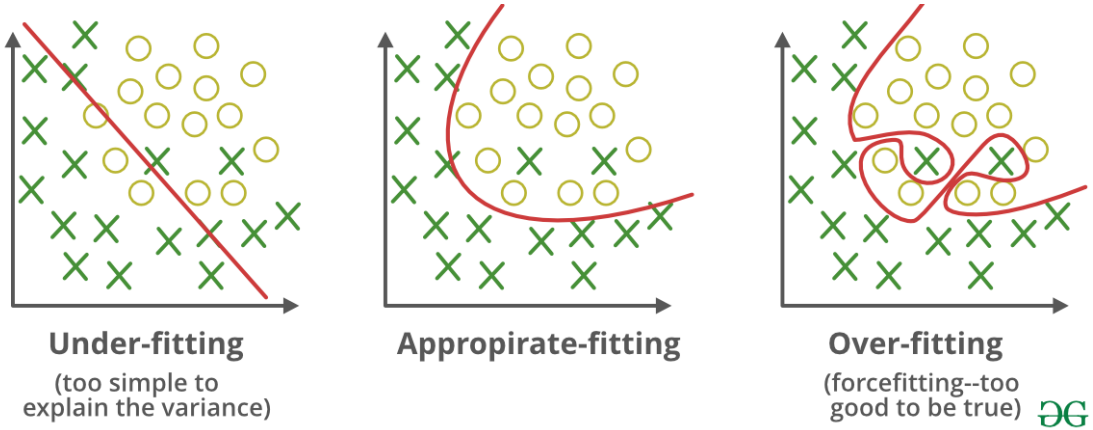
Cross-validation (overfitting and underfitting)

Cross-validation (overfitting and underfitting)

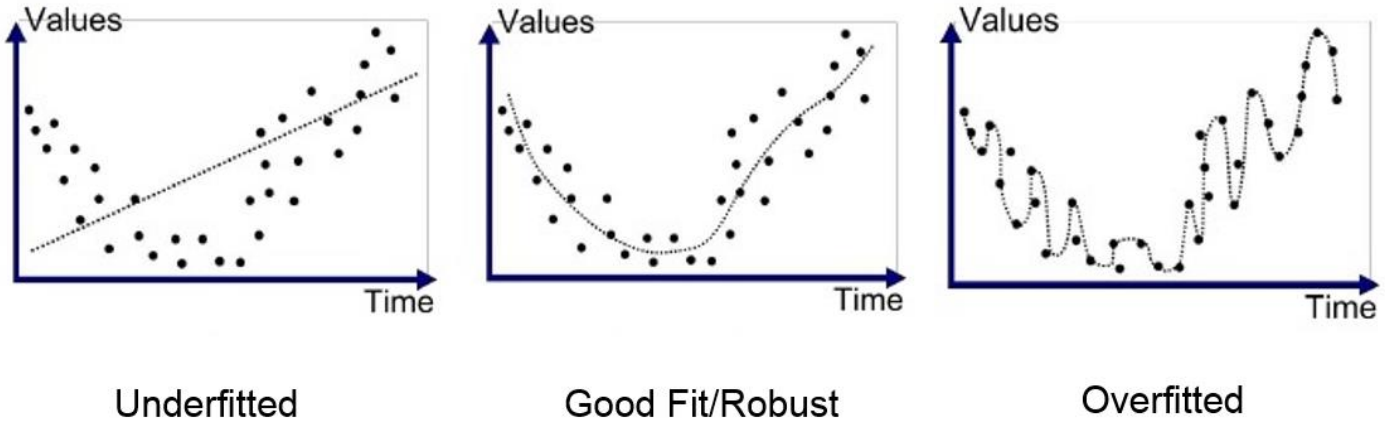
Given labelled 'training' data $\mathcal{D} = \{(\mathbf{y}^{(1)}, \mathbf{x}^{(1)}), (\mathbf{y}^{(2)}, \mathbf{x}^{(2)}), \dots, (\mathbf{y}^{(n_d)}, \mathbf{x}^{(n_d)})\} = \{(\mathbf{X}, \mathbf{Y})\}$

Weights $W^{(1)}, \dots, W^{(n_{hl})}$, with $\mathbf{X} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_d)}], \mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$ and $\mathbf{Y} = [\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(n_d)}], \mathbf{y}^{(i)} \in \{0,1\}^{n_y}$.

Classification



Regression



In “traditional” engineering overfitting is mitigated by <u>limiting the number of parameters</u>.

In ML overfitting is mitigated by <u>penalizing the magnitude of the parameters</u>.

Intuitively this can be thought as <u>penalizing</u> the gradients, i.e. <u>flexibility and rate of change</u> of models

Cross-validation (overfitting and underfitting)

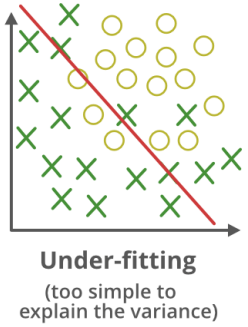
Given labelled 'training' data $\mathcal{D} = \{(\mathbf{y}^{(1)}, \mathbf{x}^{(1)}), (\mathbf{y}^{(2)}, \mathbf{x}^{(2)}), \dots, (\mathbf{y}^{(n_d)}, \mathbf{x}^{(n_d)})\} = \{(\mathbf{X}, \mathbf{Y})\}$

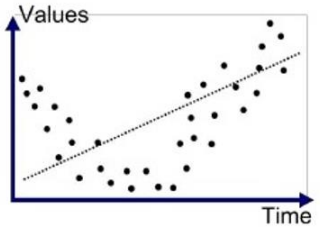
Weights $W^{(1)}, \dots, W^{(n_{hl})}$, with $\mathbf{X} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_d)}], \mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$ and $\mathbf{Y} = [\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(n_d)}], \mathbf{y}^{(i)} \in \{0,1\}^{n_y}$.

$$\min_{W^{(1)},\dots,W^{(n_{hl})}} \quad \mathcal{L}(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y}) + \lambda \sum_{i=1}^{n_w} w_i^2$$

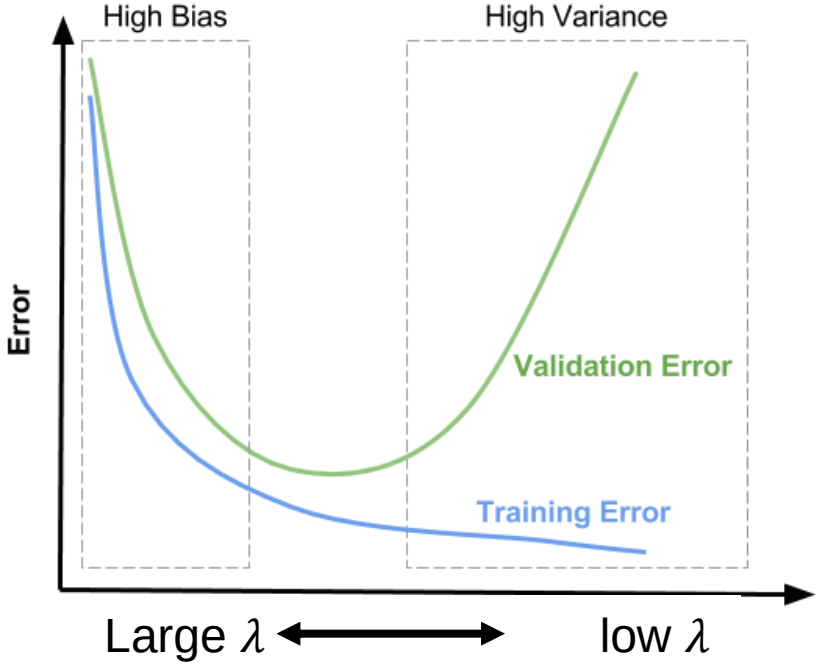
What value should λ be?

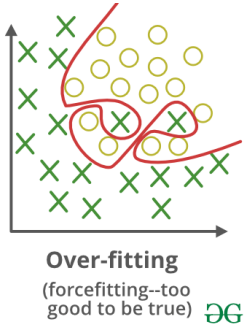
We split our data in training 80% and cross validation 20% and increase λ:

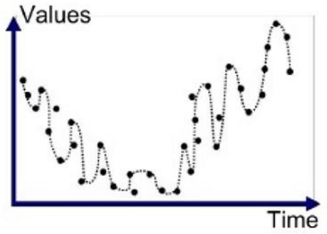




Underfitted







Overfitted

Cross-validation (overfitting and underfitting)

Given labelled 'training' data $\mathcal{D} = \{(\mathbf{y}^{(1)}, \mathbf{x}^{(1)}), (\mathbf{y}^{(2)}, \mathbf{x}^{(2)}), \dots, (\mathbf{y}^{(n_d)}, \mathbf{x}^{(n_d)})\} = \{(\mathbf{X}, \mathbf{Y})\}$

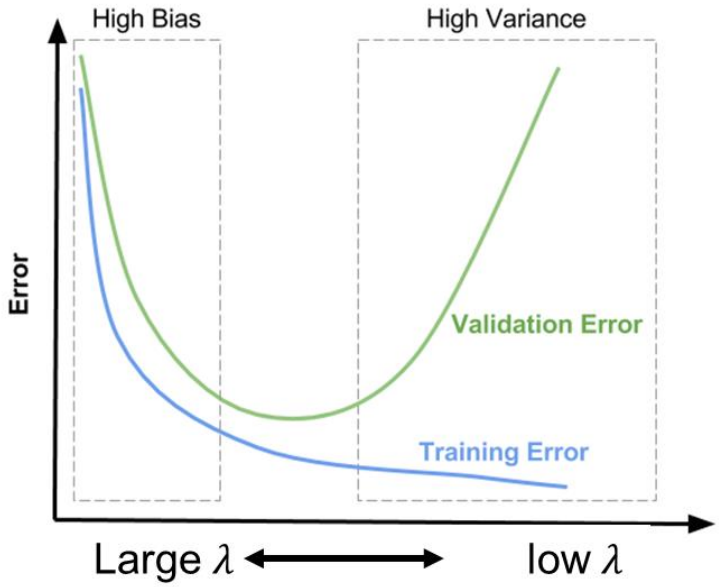
Weights $W^{(1)}, \dots, W^{(n_{hl})}$, with $\mathbf{X} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_d)}], \mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$ and $\mathbf{Y} = [\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(n_d)}], \mathbf{y}^{(i)} \in \{0,1\}^{n_y}$.

$$\min_{W^{(1)},\dots,W^{(n_{hl})}} \quad \mathcal{L}(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y}) + \lambda \sum_{i=1}^{n_w} w_i^2$$

What value should λ be?
We split our data in training 80% and cross validation 20% and increase λ:

Cross validation procedure

1. **Start** with a low (or zero) value for λ
2. **Solve** $\min_{W^{(1)},\dots,W^{(n_{hl})}} \quad \mathcal{L}(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y}) + \lambda \sum_{i=1}^{n_w} w_i^2$
3. **Increase** λ
4. **Iterate** for several values of λ
5. **Plot/choose** adequate model



Cross-validation (overfitting and underfitting)

Given labelled 'training' data $\mathcal{D} = \{(\mathbf{y}^{(1)}, \mathbf{x}^{(1)}), (\mathbf{y}^{(2)}, \mathbf{x}^{(2)}), \dots, (\mathbf{y}^{(n_d)}, \mathbf{x}^{(n_d)})\} = \{(\mathbf{X}, \mathbf{Y})\}$

Weights $W^{(1)}, \dots, W^{(n_{hl})}$, with $\mathbf{X} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_d)}], \mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$ and $\mathbf{Y} = [\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(n_d)}], \mathbf{y}^{(i)} \in \{0,1\}^{n_y}$.

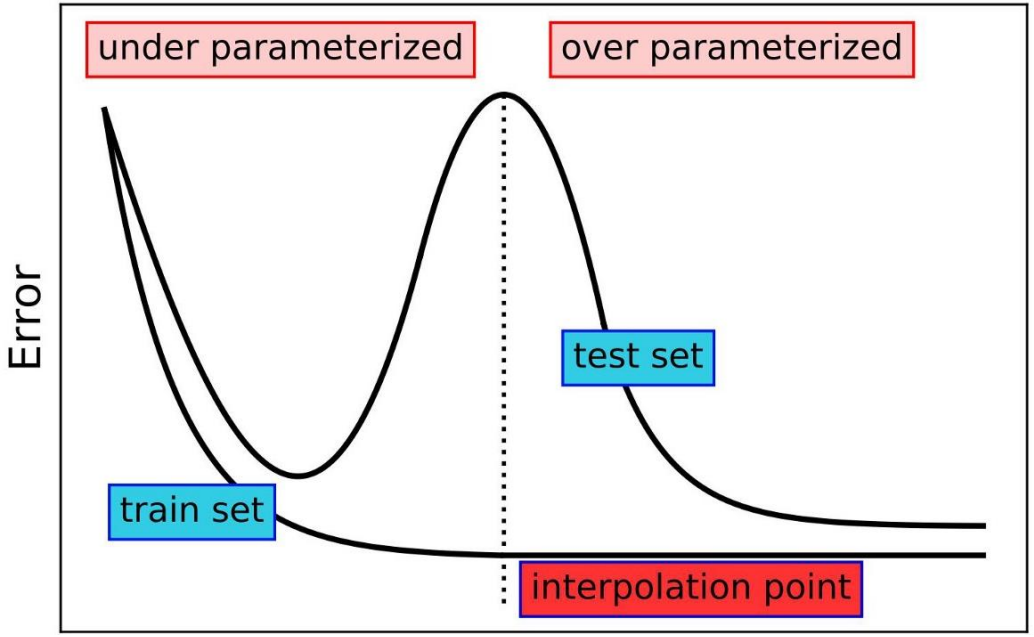
$$\min_{W^{(1)},\dots,W^{(n_{hl})}} \mathcal{L}(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y}) + \lambda \sum_{i=1}^{n_w} w_i^2$$

What value should λ be?
We split our data in training 80% and cross validation 20% and increase λ:

Cross validation procedure

1. **Start** with a low (or zero) value for λ
2. **Solve** $\min_{W^{(1)},\dots,W^{(n_{hl})}} \mathcal{L}(W^{(1)}, \dots, W^{(n_{hl})}; \mathbf{X}, \mathbf{Y}) + \lambda \sum_{i=1}^{n_w} w_i^2$
3. **Increase** λ
4. **Iterate** for sever
5. **Plot/choose** ad

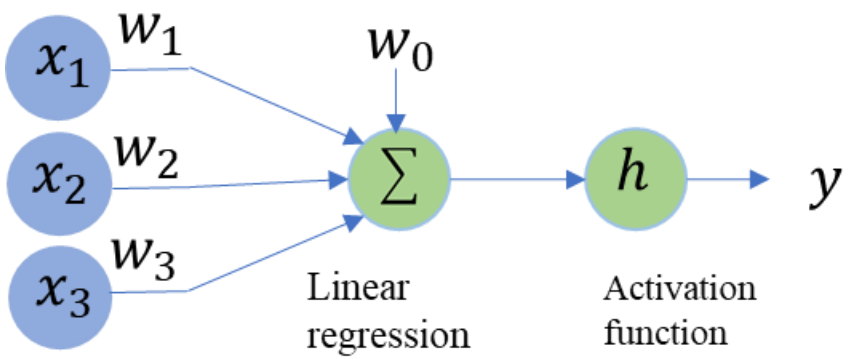
In the domain of very large datasets and very large models: 'mainstream' DL. This no longer happens. This effect is called 'double descent'.



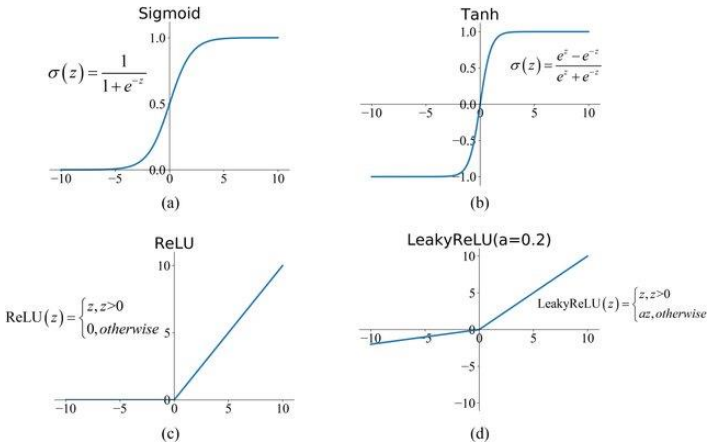
Summary

Summary – Neural Networks

A **linear regression** is contained in every neuron in a neural network.

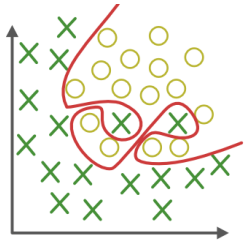


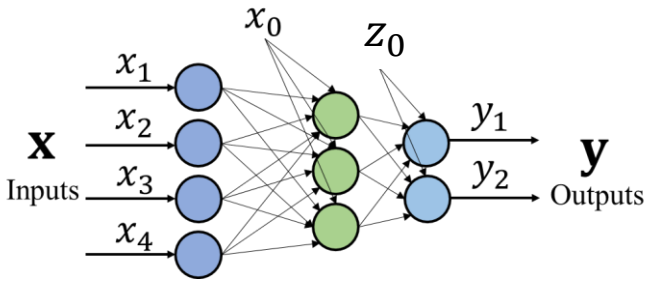
Neural networks use **activation functions** as nonlinear transformations of their inputs.

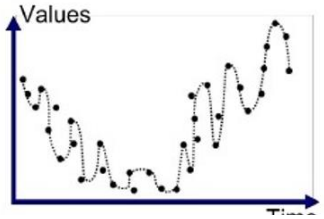


NNs are **overparametrised** functions which are **'trained' through optimisation**.

$$\min_{W^{(1)},\dots,W^{(n_{hl})}} \mathcal{L}\left(W^{(1)},\dots,W^{(n_{hl})};\mathbf{X},\mathbf{Y}\right)$$

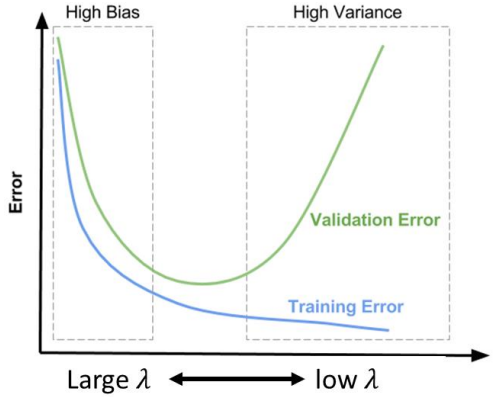






To address overfitting we can use cross-validation

$$\min_{W^{(1)},\dots,W^{(n_{hl})}} \mathcal{L}\left(W^{(1)},\dots,W^{(n_{hl})};\mathbf{X},\mathbf{Y}\right) + \lambda \sum_{i=1}^{n_w} w_i^2$$



Neural Networks – further resources

In this lecture we only covered the basics of Neural Networks. Some further resources:

- Specialization on **Deep Learning**: https://www.coursera.org/specializations/deep-learning
- NNs for **Images**: https://www.coursera.org/learn/convolutional-neural-networks
- NNs for **time series**: https://www.coursera.org/learn/nlp-sequence-models
- **Book** on Deep Learning: https://www.deeplearningbook.org/

Software:

Most widely used NN software are PyTorch, JAX and Keras+Tensorflow

For consistency, in this course we will be using scikit-learn.